

Week 4 Assignment – Tell The Story Of Objects

Business Case: Online Event Ticketing System

Group Members: Aeen, Stan

Part 1: Group Task – Conceptual Class Diagram Elaboration

Introduction

In Week 3, we created a first version of our domain model for the Online Event Ticketing System. For Week 4, we refined it by keeping only the essentials, adding clear attributes, and specifying multiplicities and business rules. We kept it conceptual (the “what”), not technical (the “how”), so it’s easy to read and use in class.

Conceptual Classes

- **Customer** — customerId, name, email, phoneNumber, address
- **Order** — orderId, orderDate, totalAmount, status, paymentMethod
- **Ticket** — ticketId, price, seatNumber?, qrCode, status
- **Event** — eventId, title, date, category, description
- **Venue** — venueId, name, location, capacity, type
- **Payment** — paymentId, amount, status, method, paymentDate
- **Organizer** — organizerId, name, contactInfo, organizationName, role

Associations and Multiplicities

- Customer **1..*** Order
- Order **1..*** Ticket
- Ticket **1..1** Event
- Event **1..1** Venue
- Organizer **1..*** Event
- Order **1..1** Payment

These links describe the core flow: organizers create events at venues; customers place orders, which contain tickets for events; each order has one payment.

Business Rules (constraints)

1. An order must be paid before tickets are confirmed (Payment.status = "Paid").
2. Venue.capacity limits the total number of tickets sold for an event date.
3. Each ticket is tied to exactly one event; seatNumber is optional for standing events.
4. An order has one payment; refunds (if needed) are handled later and are out of scope for Week 4.
5. Customer contact details (email/phone) are required for delivery of e-tickets.

Explanation

We simplified the Week 3 model to keep only the most important concepts so the diagram stays readable. **User** was narrowed to **Customer** and **Organizer** because those are the two roles we actually need here: customers buy tickets, organizers create events. **Event** is linked to **Venue** to capture where the event happens and to support a capacity constraint. **Ticket** connects orders to events (one ticket = one event occurrence) and optionally includes a seat number for seated shows. **Order** and **Payment** are modeled one-to-one to make the transaction flow easy to understand and track.

We assigned a small, meaningful set of attributes to every class so the model looks realistic without dropping into implementation details. The multiplicities reflect real situations (a customer can have many orders; an order can contain many tickets; each ticket is for one event). The business rules make sure the model can't describe impossible states (like selling more tickets than the venue can handle, or un-paid orders generating valid tickets).

Overall, this conceptual class diagram is a clear continuation of our Week 3 work. It keeps the domain story simple, covers the main objects and their relationships, and gives us a solid base for Week 5 when we'll go deeper into more advanced modeling.

